

Reference: <https://developer.chrome.com/docs/devtools/overview>

Chrome DevTools is a powerful set of web development tools integrated into Google Chrome that allows developers to inspect, debug, and optimize web applications.

1. Getting Started with Chrome DevTools

To open Chrome DevTools:

1. **Right-click** on any webpage element and select **Inspect**.
 2. Or, press **F12** (Windows) / **Cmd + Option + I** (Mac).
 3. You can also access it through **Menu > More tools > Developer tools**.
-

2. The DevTools UI Overview

The DevTools interface is organized into multiple panels, each designed to assist with different aspects of web development:

- **Elements:** Inspect and modify HTML/CSS.
 - **Console:** Execute JavaScript and log data.
 - **Sources:** View and debug JavaScript.
 - **Network:** Monitor network requests.
 - **Performance:** Record and analyze runtime performance.
 - **Memory:** Profile memory usage.
 - **Application:** Manage storage, caches, and service workers.
 - **Security:** Check security information.
 - **Lighthouse:** Run audits to analyze and improve page performance.
-

3. Elements Panel

The **Elements panel** allows you to inspect and modify HTML and CSS in real-time.

- **Inspect Elements:** Hover over the HTML structure to see the corresponding visual element highlighted on the page.
 - **Editing HTML and CSS:** Double-click any element or CSS property to edit.
 - **CSS Box Model:** View the box model (margin, border, padding) in the right sidebar.
 - **Styles Tab:** Modify CSS rules and see changes immediately.
 - **Layout Tab:** View grid and flexbox layout options.
 - **Event Listeners:** View active JavaScript event listeners on elements.
-

4. Console Panel

The **Console panel** is used to execute JavaScript, debug, and view error messages.

- **Logging:** Use `console.log()`, `console.error()`, `console.warn()` for debugging.
 - **Executing Commands:** You can write and execute JavaScript expressions directly.
 - **Filtering:** Filter messages by type (e.g., log, error, warning).
 - **Access Elements:** Use `$0`, `$1` to reference elements selected in the Elements panel.
-

5. Sources Panel

The **Sources panel** is a comprehensive JavaScript debugging environment.

- **File Navigator:** View and open all resources associated with the page.
 - **Breakpoints:** Set breakpoints by clicking on line numbers. You can use various types, such as line breakpoints, conditional breakpoints, and DOM breakpoints.
 - **Watch Expressions:** Monitor specific variables or expressions.
 - **Call Stack:** View the call stack to trace function calls.
 - **Scope:** Inspect local and global variables.
 - **Step Through Code:** Use **Step Over** (F10), **Step Into** (F11), and **Step Out** (Shift+F11) to navigate.
-

6. Network Panel

The **Network panel** is essential for analyzing and troubleshooting network requests.

- **Monitoring Requests:** Shows all requests, their status, and response times.
 - **Throttling:** Simulate slower network speeds to test loading under various conditions.
 - **Inspecting Headers and Data:** View request/response headers, cookies, and payloads.
 - **XHR and Fetch:** Filter and view XMLHttpRequest and Fetch API calls.
 - **Performance Analysis:** Look at resource loading times and prioritize optimization efforts.
-

7. Performance Panel

The **Performance panel** helps you record and analyze the runtime performance of your page.

- **Record Profile:** Click "Record" to capture page interactions and load time.
- **Flame Chart:** Shows the time spent in each function during the recording.
- **Frame Analysis:** Helps identify bottlenecks in rendering and paint times.
- **Screenshots:** Capture screenshots during the recording for visual reference.
- **Analyze Metrics:** View FPS, CPU load, and memory usage during the recording.

8. Memory Panel

The **Memory panel** is useful for detecting and managing memory leaks.

- **Heap Snapshot:** Take snapshots of memory usage at different times to identify memory leaks.
 - **Allocation Sampling:** See which functions are allocating the most memory.
 - **Allocation Timeline:** Track memory allocations over time.
 - **Identify Leaks:** Use the retained size and detached DOM tree features to find potential memory leaks.
-

9. Application Panel

The **Application panel** allows you to manage and inspect web storage and resources.

- **Storage:** View and manage Local Storage, Session Storage, IndexedDB, Web SQL, and cookies.
 - **Service Workers:** Inspect active service workers and caches.
 - **Cache:** View cached assets in Application Cache and Cache Storage.
 - **Manifest:** View PWA manifest and check compatibility.
 - **Background Sync & Push:** Test and debug background synchronization and push notifications.
-

10. Security Panel

The **Security panel** gives information on the security of the website.

- **Certificates:** Check SSL certificates for validity and issues.
 - **Security Warnings:** Identify mixed-content warnings or insecure resources.
 - **Security Status:** Verify if the site is fully secure and encrypted.
-

11. Lighthouse Panel

The **Lighthouse panel** provides an automated analysis of the web page's performance, accessibility, SEO, and more.

- **Generate Reports:** Run audits to generate reports.
- **Review Recommendations:** See specific recommendations to improve performance, accessibility, and best practices.
- **Performance Score:** Get a performance score and learn how to improve page speed.

12. More Tools and Customization

The DevTools interface can be customized and expanded with additional tools and shortcuts:

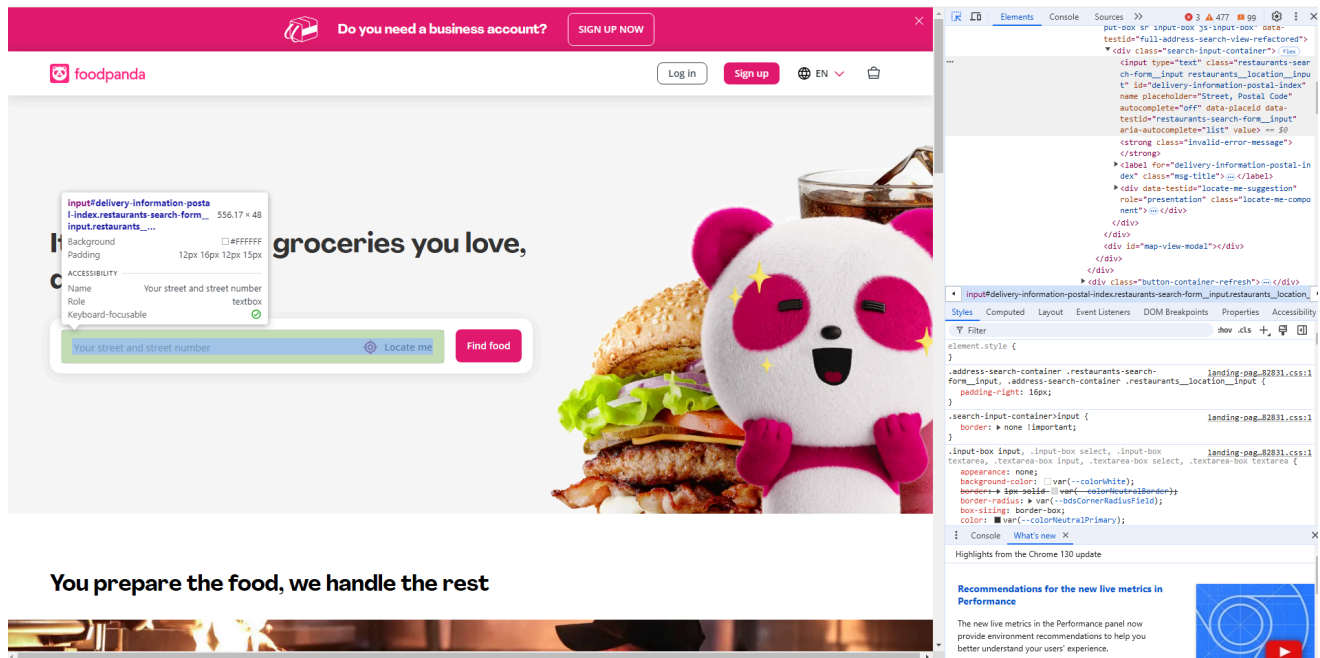
- **More Tools Menu:** Access tools like **Coverage**, **Rendering**, and **Animations**.
- **Dock Position:** Adjust the DevTools window position (bottom, side, or separate window).
- **Device Mode:** Simulate mobile and tablet devices with device emulation.
- **Shortcuts:** Learn keyboard shortcuts to speed up your workflow.

Tips and Best Practices

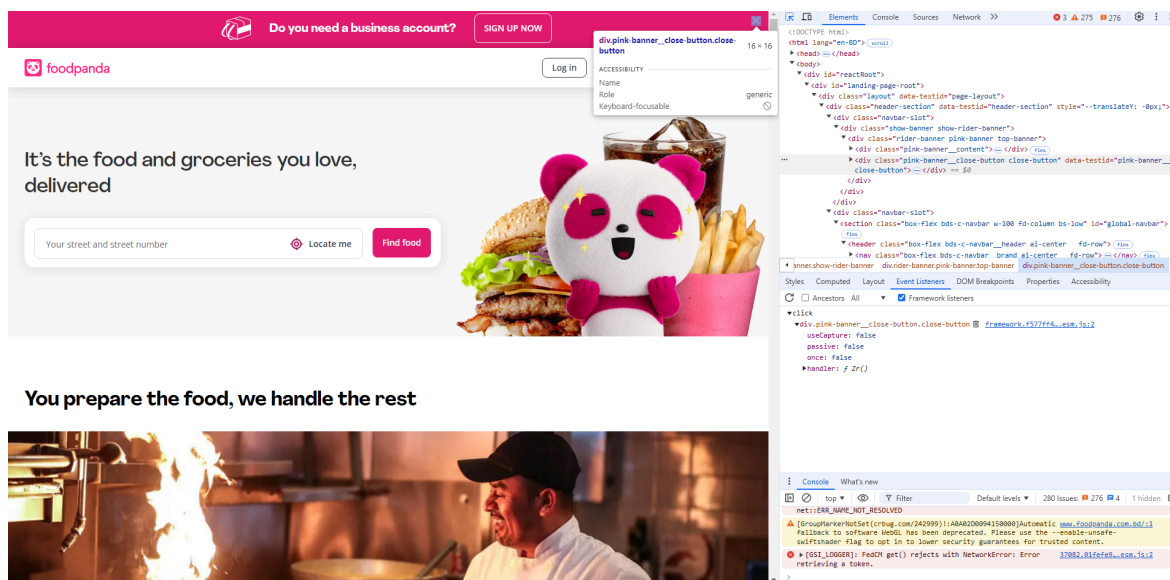
- **Use Console for Quick Debugging:** Use `console.log()` and similar commands to track variable values.
 - **Throttling:** Always test on simulated slower networks to ensure good user experience.
 - **Use Responsive Design Mode:** Test page responsiveness on various devices.
 - **Use Persistent Console Logs:** In Console settings, enable "Preserve log" to keep logs between page reloads.
-

Lab Tasks

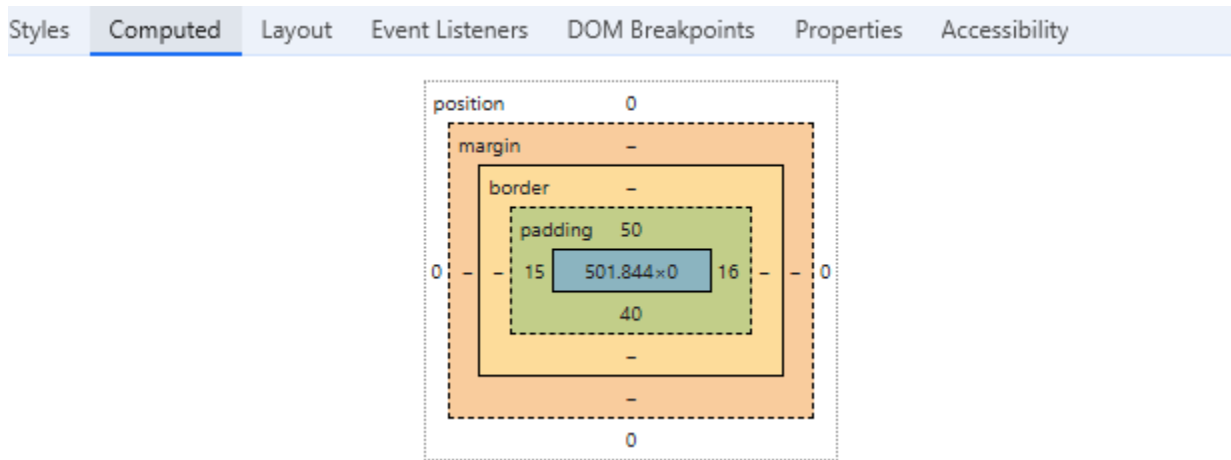
1. Each group will choose one existing website for experimentation which is similar to their selected project topic. For example: <https://www.foodpanda.com.bd/>
2. Open DevTools (Ctrl + Shift + C). Click on an element.



3. Use the inspect() method in the console, such as inspect(document.body). See the Command-Line API for information on using inspect.
4. Navigate through the DOM structure using either mouse or keyboard. To expand a collapsed node, click the node. As you navigate, the Elements panel shows a breadcrumb trail in the bottom bar.
5. View element event listeners.

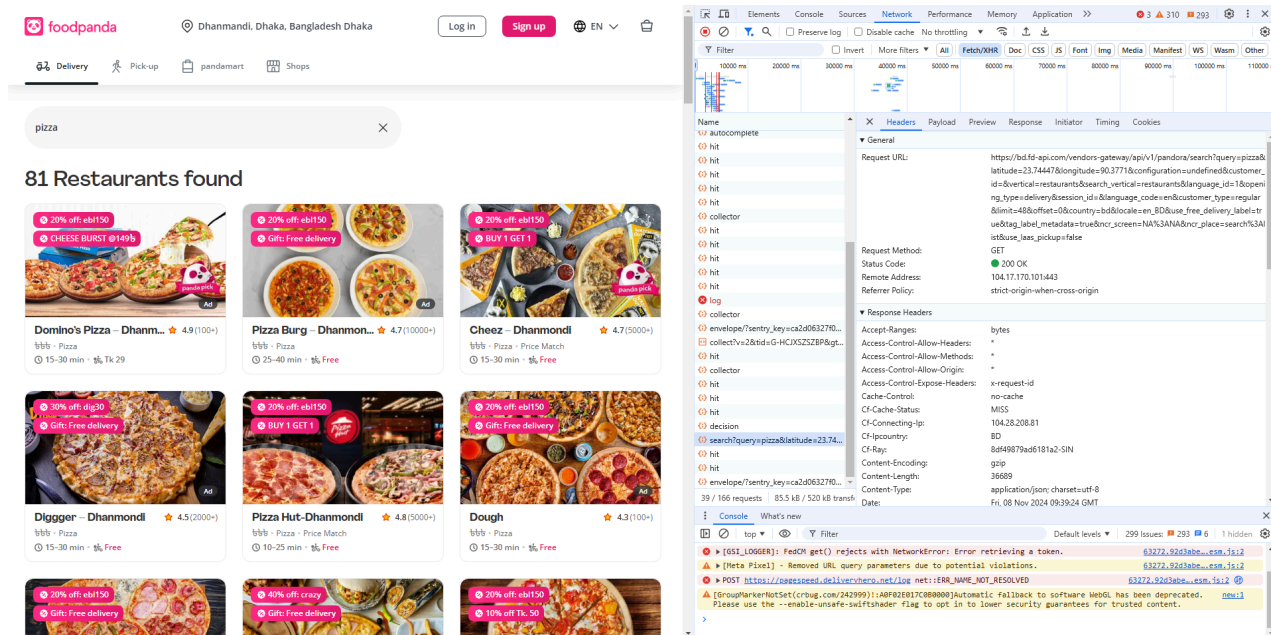


6. Change the values on the Metrics pane of an element.



7. Now, Type “Dhanmondi” in the search bar and click the “Find Food” button. Click on “Find Restaurants. Type “Pizza” in the search option.

8. Check the Network tab. User filter: Fetch/XHR. Click on the “search?query=pizza...”. Check the Header, Payload and Response tab.

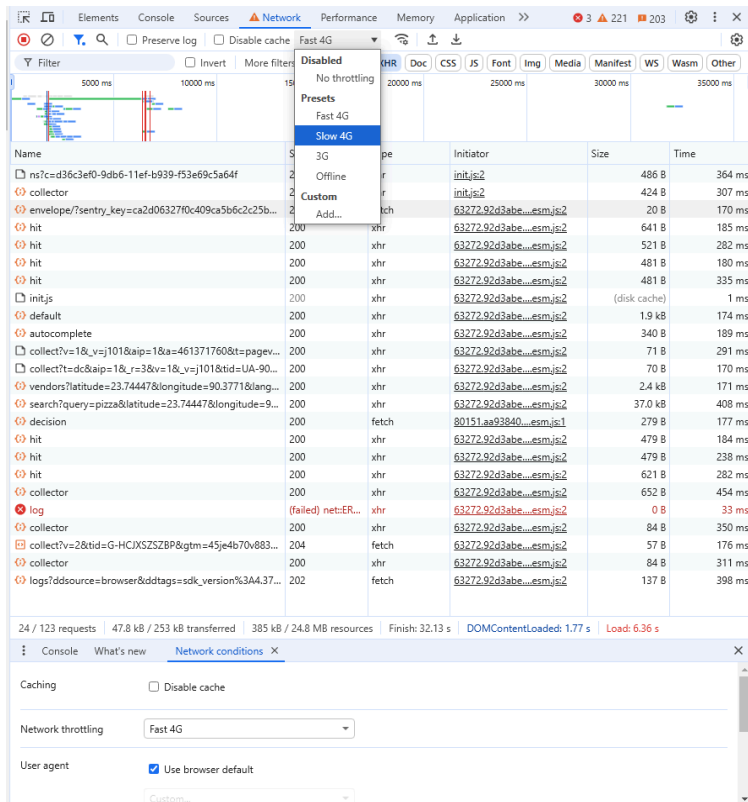


HTTP: https://www.w3schools.com/whatis/whatis_http.asp

HTTP Request Methods: https://www.w3schools.com/tags/ref_httpmethods.asp

JSON: https://www.w3schools.com/whatis/whatis_json.asp

9. Change the throttling. Check the number of requests and load time in the bottom.

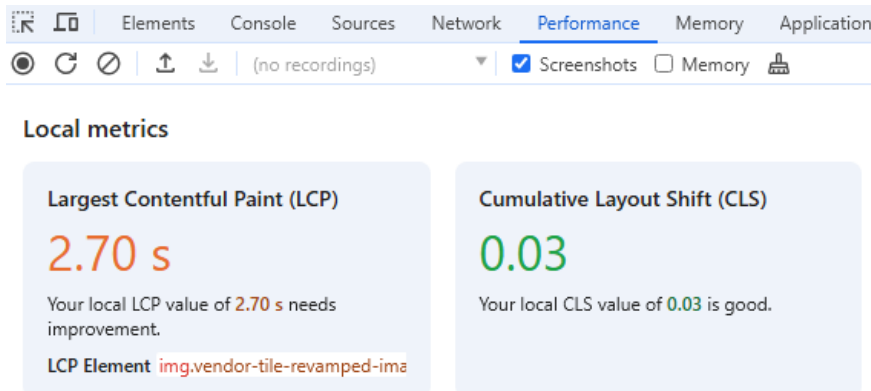


10. Right click on the reload button and select “Empty Cache and hard reload”. Now, Check the Performance tab.

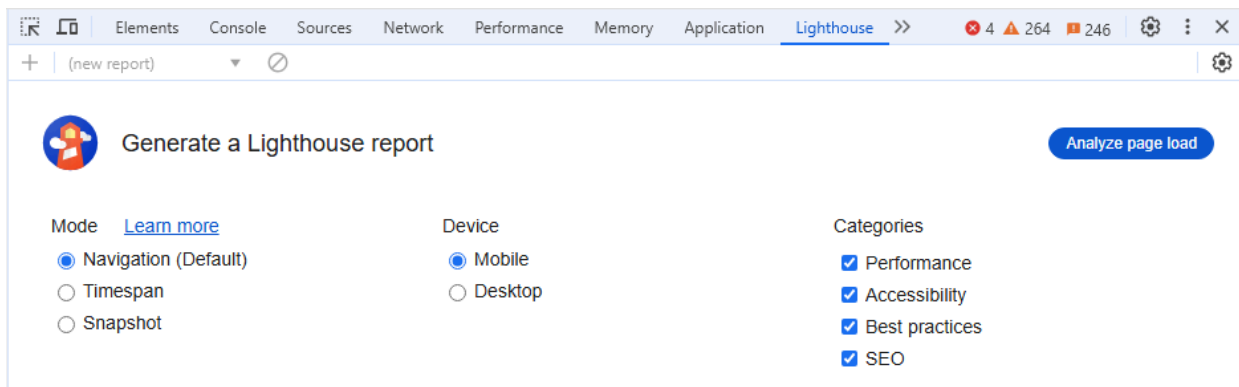
Largest contentful paint (LCP) provides a snapshot of how long a page's main content takes to become visible to users. A good LCP is **less than 2.5 seconds**. LCP between 2.5 and 4 seconds is considered "needing improvement", and above 4 seconds is considered poor.

Cumulative Layout Shift (CLS) is a metric that measures how much a webpage shifts unexpectedly while it's loading and being rendered. To improve CLS, you can reserve space for images, videos, and iframes. You can also use the CSS property aspect-ratio to set a dynamic width based on the screen size. Good CLS values are **0.1 or less**. Poor values are greater than 0.25.

Repeat this experiment with a different webpage: <https://dlrs.gov.bd/>



11. Go to the Lighthouse tab and click the Analyze page load button.



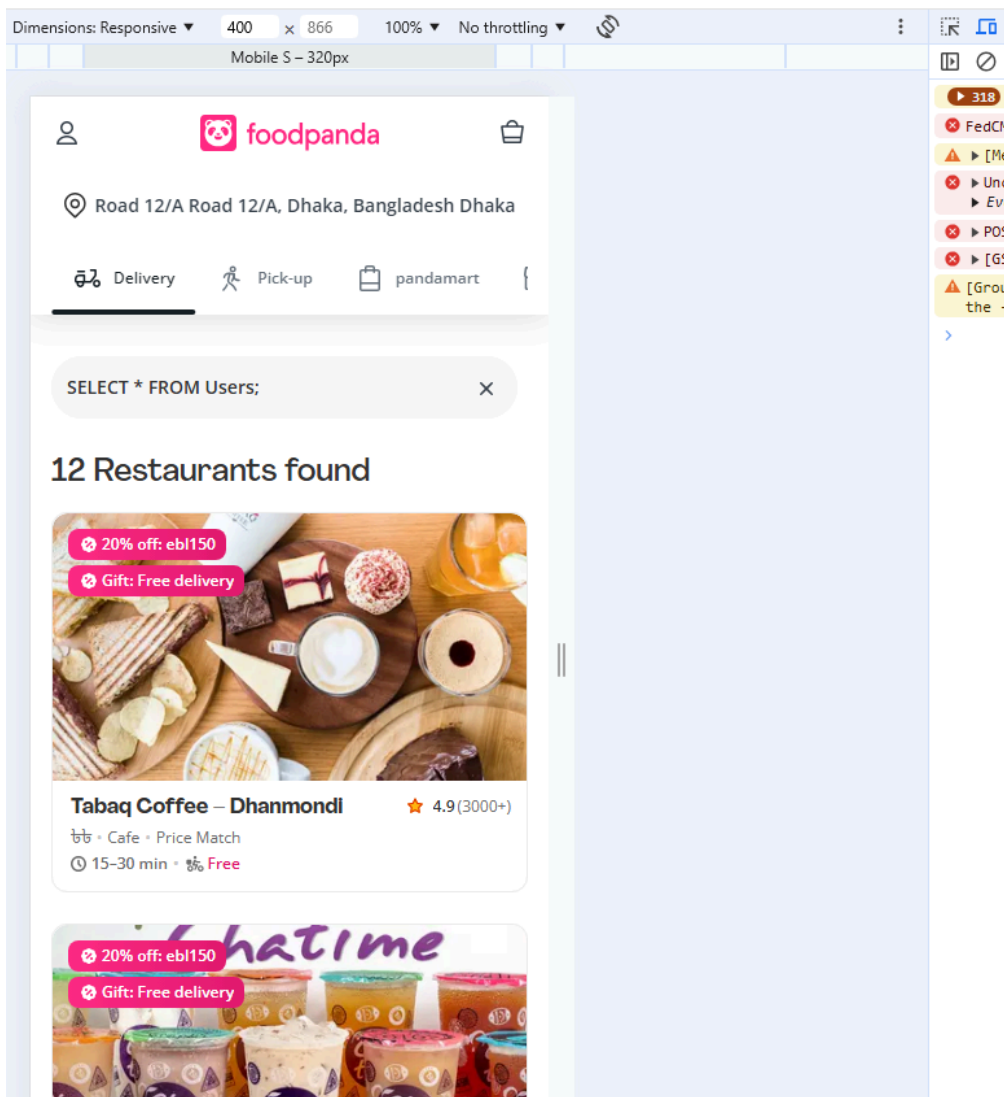
0 to 49 (red): Poor

50 to 89 (orange): Needs Improvement

90 to 100 (green): Good

Do this same experiment with another website and compare: <https://dlrs.gov.bd/>

12. Toggle device toolbar (Ctrl + Shift + M). View the page in different viewports.



13. Go to the Console tab. Console is useful because-

- Making sure that code is executing in the right order.
- Inspecting the values of variables at a certain moment in time.

14. Execute the following codes in console:

```
console.log("Testing console");

document.title = "This page is hacked";
document.body.style.backgroundColor = "red";

const h1 = document.getElementById("rlp-vertical-switcher");
if (h1) {
  console.log(h1.textContent);
} else {
  console.warn("Element with id 'rlp-vertical-switcher' not found.");
}

const artists = [
  { first: "René", last: "Magritte" },
  { first: "Chaim", last: "Soutine" },
  { first: "Henri", last: "Matisse" }
];

console.table(artists);
```